High Performance Programming [COMTEK3, & ESD3]
Lecture 1: Introduction to Parallel Processing 02/2025

Instruction: *The exercise problems are designed to aid your understanding of the concepts covered during the lecture. They are intended to be discussed and solved in a group of* $3-6$ *students. The most important part of the exercise is the* ***learning process****. It is strongly recommended that you avoid the usage of LLMs such as chatGPT and similar AI models to generate solutions* ***without trying to solve the problems first****.*

Exercise 1: **Parallel Computing Models and Scheduling**

Sketch the binary tree models and specify a possible schedule for the summation of 16 numbers - $a_1, a_2, \cdots, a_{16}$ using

(a) a single processor (sequential computation)

(b) 4 processors

(c) 5 processors

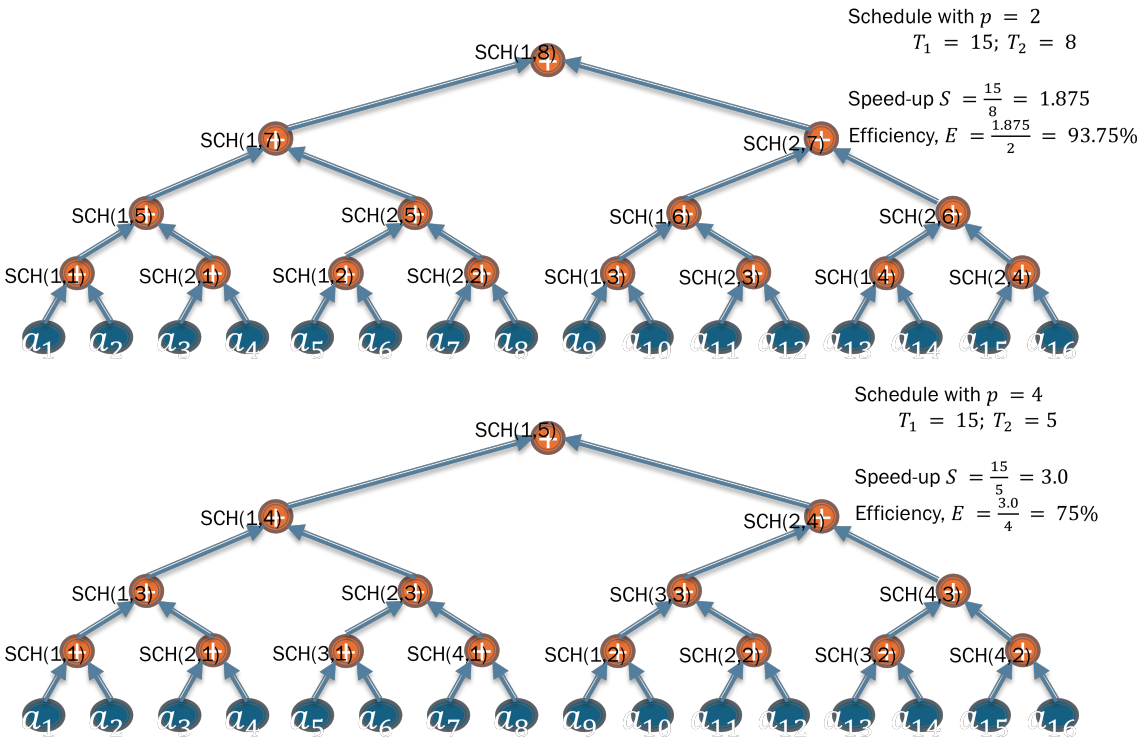
(d) 8 processors

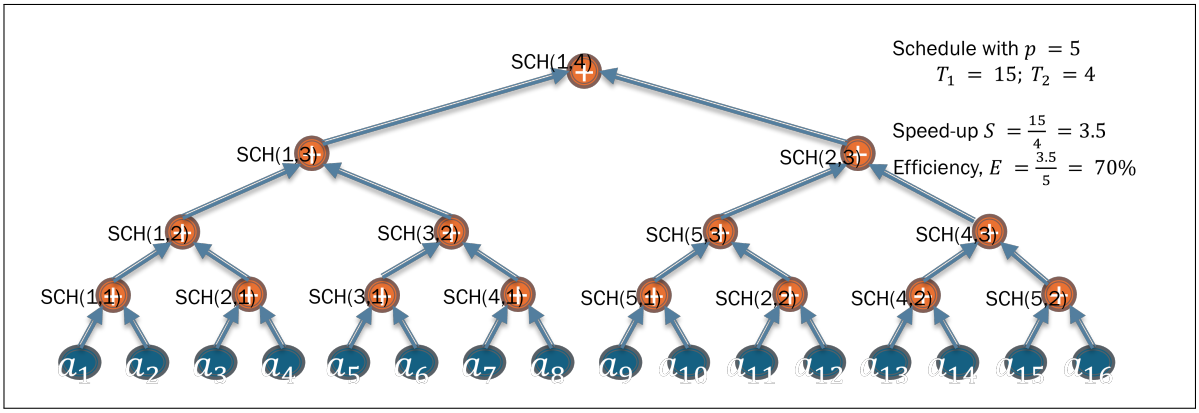
(e) 16 processors

(f) Discuss the potential speedup from your schedule in b - e.

(g) Determine the efficiency of your schedule in b - e.

Solution: Example model along with schedule and calculation of speedup and efficiency with $p = 2$, $p = 4$ and $p = 5$ are shown below. Others can be obtained by following the same step.





Exercise 2: **Parallel Computing Models and Performance**

Consider the problem of finding the value of the polynomial:

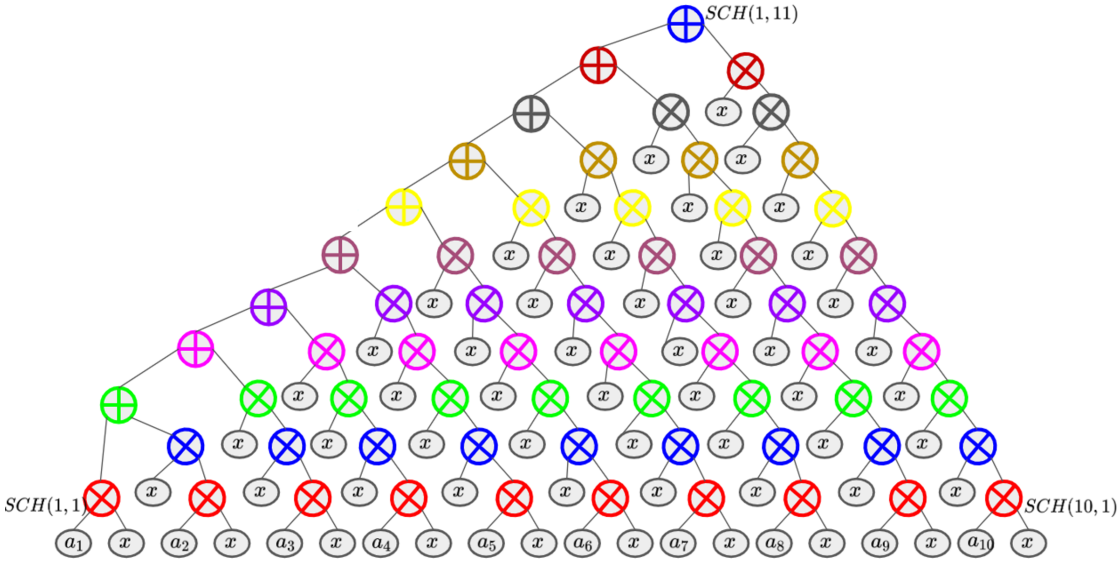
$$f(x) = \sum_{i=0}^{10} a_i x^i$$

(a) Assuming that ten processors are available to work in parallel, draw the binary tree model of the computation.

(b) What is the number of processors for achieving maximum speedup and efficiency?

Solution:

(a) The binary tree model for the computation of $f(x) = \sum_{i=1}^{10} a_i x^i$ with the most trivial schedule is shown in the figure below. Note that the schedule can be made more efficient by avoiding repeated multiplications to calculate the different powers of x.



(b) From the model, we observe:

$$T_1 = W_1 = W_10 = 64$$

$$T_10 = 11$$

$$\text{Speed}, S = 64/11 = 5.82$$

$$\text{Efficiency}, E = 5.82/10 = 58.2\%$$

Exercise 3: **Parallel Processing Effectiveness**

An image processing application problem is characterized by 12 unit-time tasks:

1. an input task that must be completed before any other task can start and consumes the entire bandwidth of the single-input device available,
2. 10 completely independent computational tasks, and
3. an output task that must follow the completion of all other tasks and consumes the entire bandwidth of the single-output device available.

Assume the availability of one input and one output device throughout.

(a) What is the maximum speed-up that can be achieved for this application with two processors?

(b) What is an upper bound on the speed-up with parallel processing?

(c) How many processors are sufficient to achieve the maximum speed-up derived in part (b)?

(d) What is the maximum speed-up in solving five independent instances of the problem on two processors?

(e) What is an upper bound on the speed-up in parallel solution of 100 independent instances of the problem?

(f) How many processors are sufficient to achieve the maximum speed-up derived in part (f)?

(g) What is an upper bound on the speed-up, given a steady stream of independent problem instances?

Solution: (a) The problem consists of:

- 1 input task (1 unit-time),
- 10 independent computational tasks (1 unit-time each),
- 1 output task (1 unit-time).

With a single processor, the total execution time is:

$$T_1 = 1 + 10 + 1 = 12 \tag{1}$$

With two processors, the 10 computational tasks can be executed in parallel, taking:

$$T_2 = 1 + \frac{10}{2} + 1 = 1 + 5 + 1 = 7 \tag{2}$$

The speed-up is:

$$S_2 = \frac{T_1}{T_2} = \frac{12}{7} \approx 1.71 \tag{3}$$

(b) With infinite processors, the computational tasks take only 1 unit time, leading to:

$$T_\infty = 1 + 1 + 1 = 3 \tag{4}$$

The maximum theoretical speed-up is:

$$S_{\max} = \frac{T_1}{T_\infty} = \frac{12}{3} = 4 \tag{5}$$

(c). To achieve $S_{\max} = 4$, the 10 computational tasks must be completed in 1 unit-time, requiring at least 10 processors.

(d). Each instance runs in $T_2 = 7$ units. With two processors, the indepedent instances execute sequentially:

$$T_{5,2} = 5 \times 7 = 35 \tag{6}$$

Serial execution takes:

$$T_{5,1} = 5 \times 12 = 60 \tag{7}$$

Thus, the speed-up is:

$$S_{5,2} = \frac{T_{5,1}}{T_{5,2}} = \frac{60}{35} \approx 1.71 \tag{8}$$

(e). With infinite processors, all instances can run concurrently in 3 units, so:

$$T_{100,\infty} = 3 \tag{9}$$

Serial execution takes:

$$T_{100,1} = 100 \times 12 = 1200 \tag{10}$$

Thus, the theoretical speed-up is:

$$S_{100,\infty} = \frac{1200}{3} = 400 \tag{11}$$

(f).To achieve $S_{100,\infty} = 4$, each of the 100 instances must have 10 processors for computations. Thus, a total of:

$$100 \times 10 = 1000 \tag{12}$$

are required.

(g). For an infinite sequence of problems, the pipeline is limited by the longest stage. Since each instance requires 3 units in the best case:

$$S_{steady} = 4 \tag{13}$$

This is the upper bound given the input and output constraints.

Exercise 4: **Amdahl's Law**

Amdahl's law can be applied in contexts other than parallel processing. Suppose that a numerical application consists of 20% floating-point and 80% integer/control operations (these are based on operation counts rather than their execution times). The execution time of a floating-point operation is three times as long as other operations. We are considering a redesign of the floating-point unit in a microprocessor to make it faster.

(a) Formulate a more general version of Amdahl's law in terms of selective speed-up of a portion of a computation rather than in terms of parallel processing.

(b) How much faster should the new floating-point unit be for 25% overall speed improvement?

(c) What is the maximum speed-up that we can hope to achieve by only modifying the floating-point unit?

Solution:

(a) Amdahl's Law describes the theoretical speed-up in performance when a portion of a computation is improved. If a fraction α of a program's execution time is accelerated by a factor of β, then the overall speed-up S is given by:

$$S = \frac{1}{(1-\alpha) + \frac{\alpha}{\beta}} \tag{14}$$

where:

- α is the fraction of execution time affected by the improvement,
- β is the speed-up factor for the affected portion.

(b) The current execution time includes:

- Floating-point operations: 20% of operations but take 3× longer than integer operations.
- Integer/control operations: 80% of operations.

Thus, the actual fraction of the total time spent on floating-point operations is:

$$\frac{0.2 \times 3}{0.2 \times 3 + 0.8} = \frac{0.6}{1.4} \approx 0.4286 \tag{15}$$

We seek the speed-up factor β such that:

$$\frac{1}{(1 - 0.4286) + \frac{0.4286}{\beta}} = 1.25 \tag{16}$$

Solving for β:

$$\begin{aligned} 1.25 &= \frac{1}{0.5714 + \frac{0.4286}{S}} \\ 0.5714 + \frac{0.4286}{\beta} &= \frac{1}{1.25} \\ 0.5714 + \frac{0.4286}{\beta} &= 0.8 \\ \frac{0.4286}{\beta} &= 0.2286 \\ \beta &= \frac{0.4286}{0.2286} \approx 1.875 \end{aligned}$$

Thus, the floating-point unit should be 1.875× faster to achieve a 25% overall speed-up.

(c) If the floating-point unit were infinitely fast ($\beta \to \infty$), the best possible speed-up is given by:

$$S_{max} = \frac{1}{1 - 0.4286} = \frac{1}{0.5714} \approx 1.75 \tag{17}$$

RAD